PROOFLOOP

Technical Architecture & Implementation Report

Google Cloud Platform • Flutter • FastAPI • Evidence-grounded AI

Reference design for a human-in-the-loop document reconciliation platform

Document type	Technical architecture report
Status	Proposed reference architecture; validate service/model availability before deployment
Target region	asia-southeast1 (Singapore), subject to data-residency and service-availability checks
Primary principle	Code calculates; AI interprets documents and drafts explanations; people decide

1. Executive summary

ProofLoop is a case-based evidence review and reconciliation system. It accepts documents, extracts traceable claims, applies deterministic rules to identify discrepancies, and presents findings with source evidence for an authorized reviewer. The system is designed so that generative AI does not perform arithmetic, silently invent evidence, or make final case decisions.

The recommended implementation uses a Flutter client for web and mobile; Firebase Authentication and App Check for client identity and app attestation; FastAPI on Cloud Run for the application API; Cloud Storage for evidence and generated reports; Pub/Sub for asynchronous processing; Cloud Run workers for orchestration; Document AI for OCR; Vertex AI Gemini for schema-constrained extraction and plain-language explanation; Cloud SQL for PostgreSQL for relational case data and audit history; and Cloud Build/Artifact Registry for delivery.

The architecture deliberately separates the request path from long-running document processing. It also separates machine-produced claims, rule-derived findings, and human review decisions into distinct records and lifecycle states.

2. Goals, scope, and non-goals

2.1 Goals

- Support case creation, secure evidence upload, asynchronous processing, source-linked findings, reviewer corrections, and PDF export.
- Preserve provenance from every extracted value back to a document, page, and source span or bounding box where available.
- Make arithmetic and reconciliation reproducible in ordinary application code with testable rules.
- Provide an auditable human review workflow: confirm, correct, dismiss, annotate, and export.
- Deploy API and worker services on Cloud Run with independent scaling and least-privilege identities.
- Offer a browser-accessible Flutter web build for demonstrations alongside mobile builds.

2.2 Non-goals for the first release

- No autonomous approval, rejection, payment, enforcement, or other consequential case decision by an AI model.
- No open-ended agent with write access to the database, storage, or external systems.
- No claim that OCR or model output is always correct; uncertain or ungrounded values must be reviewable.
- No assumption that all services or model versions are available in the selected region until checked against current GCP documentation.

3. Architecture overview

Figure 1 presents the proposed logical architecture. Arrows represent principal data/control flows rather than every network call.

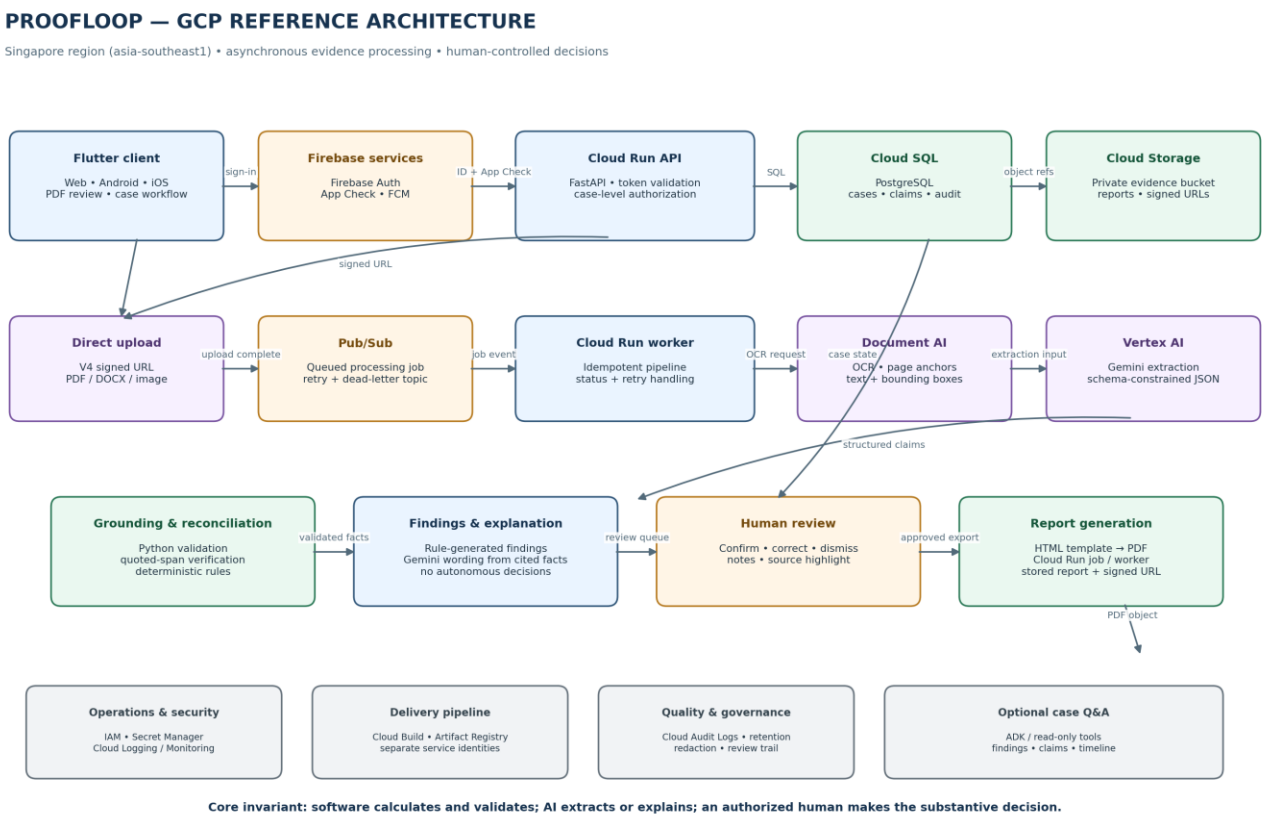


Figure 1. ProofLoop reference architecture on Google Cloud.

3.1 Main components and responsibilities

Component	Responsibility
Flutter (Dart)	Single client codebase for Flutter web, Android, and iOS; case lists, uploads, processing status, PDF evidence viewer, overlays, review forms, notes, and report download.
Firebase Authentication	User sign-in and identity tokens. The API verifies Firebase ID tokens server-side and derives the user identity from the verified token, never from a client-supplied user ID.
Firebase App Check	Adds an app-attestation signal to reduce unauthorized clients. It supplements authentication and authorization; it is not a replacement for either.
Cloud Run — FastAPI API	Synchronous application endpoints, validation, authorization, signed URL creation, case operations, review actions, status queries, and export requests.
Cloud Storage	Private buckets for original evidence, normalized/OCR artifacts if retained, and generated PDF reports. Objects are not publicly readable.
Cloud SQL for PostgreSQL	Transactional system of record for cases, evidence metadata, extracted claims, findings, review actions, job state, and audit queries.
Pub/Sub	Decouples upload/API activity from long-running processing; supports retries, delivery control, and dead-letter handling.
Cloud Run worker	Idempotent processing pipeline: extract text, call OCR/model services, ground claims, execute deterministic rules, draft explanations, persist results, and update status.

Document AI	OCR for scans and images, returning recognized text and page geometry. Parser selection should be tested against the actual document types.
Vertex AI Gemini	Structured classification/extraction and constrained plain-language explanations. Model output is treated as untrusted input and validated in code.
Cloud Build + Artifact Registry	Build/test container images, scan or validate artifacts where configured, push versioned images, and deploy services through controlled pipelines.
Secret Manager, IAM, Cloud Logging/Monitoring	Secrets, service-to-service identity and permissions, operational logs, metrics, alerting, and incident investigation.
Optional ADK case Q&A	A read-only assistant using narrowly scoped functions such as get_findings, get_claims, and get_timeline. Exclude write tools from the initial release.

4. End-to-end case lifecycle

Create case and authenticate

The client signs in through Firebase Authentication. Each API request includes the Firebase ID token and, where supported by the client platform, an App Check token. FastAPI verifies tokens, checks account status, and applies case-level authorization on every request.

Request upload authorization

The client requests an upload slot for a case and supplies metadata such as file name, declared MIME type, and size. The API checks the user's permission and issues a short-lived V4 signed URL for a specific object key and allowed method. The server must not allow the client to choose arbitrary bucket paths.

Upload evidence directly

The client uploads directly to a private Cloud Storage bucket. The API finalizes the upload only after checking object existence, size, expected metadata, and a server-computed checksum where feasible. Do not trust a checksum supplied by the client as proof of integrity.

Enqueue processing

The API writes the EvidenceFile row and an outbox/job record transactionally, then publishes a Pub/Sub message. To avoid a database-write/message-publish gap, use a transactional outbox dispatcher or a recoverable publish workflow.

Extract text and anchors

For digital PDFs and supported office documents, use deterministic parsers where practical. For scanned PDFs and images, call Document AI OCR. Normalize the result into page-level text blocks, character spans, and bounding boxes while retaining the original file.

Classify and extract claims

Send only the minimum necessary document text to Vertex AI Gemini. Use a JSON schema and explicit instructions to return typed fields, evidence quotations, page hints, confidence or uncertainty markers, and a reason for abstention. Validate every field against the schema in application code.

Ground claims

Search for the returned quote in normalized source text, with page-local matching and carefully controlled normalization. Where possible, map the matched span to OCR blocks and bounding boxes. If the quote cannot be located unambiguously, set the claim to needs_review or ungrounded rather than treating it as verified.

Run deterministic reconciliation

Python rules calculate differences, compare normalized values, enforce units and date rules, and identify missing or inconsistent evidence. Each finding stores the rule identifier/version, inputs, computed result, and evidence references so it can be reproduced.

Draft explanation

Gemini may turn a verified finding and its evidence into concise plain language. It must not introduce new facts. The explanation is labeled as machine-generated and is secondary to the underlying values, rule result, and citations.

Review and export

A reviewer sees the finding alongside the cited page and highlighted source area, then confirms, corrects, dismisses, or annotates it. Each action creates an immutable audit event. Export is generated from persisted facts and review outcomes, then saved to Cloud Storage and served through a short-lived signed URL.

5. Data architecture and suggested schema

Use PostgreSQL as the canonical application record. Store binary files in Cloud Storage and keep object references—not file bytes—in the database.

Table / entity	Important fields / purpose
users / memberships	Firebase UID, display metadata, organization or tenant, role, account status. Keep authorization roles server-managed.
cases	Case ID, tenant/owner, title, lifecycle status, created/updated timestamps, version number.
evidence_files	Evidence ID, case ID, object key, original filename, MIME type, byte size, server checksum, upload timestamp, processing status, parser version.
document_pages / text_blocks	Evidence ID, page index, normalized text, block/paragraph identifiers, bounding box, coordinate system, OCR confidence where provided.
extracted_claims	Claim ID, evidence ID, field name, raw value, normalized value, data type, quoted excerpt, page, text span, bbox, grounding status, model/version, prompt/schema version, confidence if available.
findings	Finding ID, case ID, rule ID/version, finding type, severity, input claim IDs, expected/observed values, computed difference, status, explanation, created timestamp.
review_actions	Action ID, finding ID, actor UID, action type, previous/new values, reason/note, timestamp, request/correlation ID.
processing_jobs	Job ID, evidence ID, idempotency key, attempt count, lease/status, error class, next retry time, timestamps.
reports	Report ID, case ID, object key, generated-by UID, report template/version, generation timestamp, checksum.
audit_events	Append-only security and lifecycle events including actor, action, target, timestamp, and correlation ID.

5.1 Data integrity rules

- Use UUIDs or similarly unguessable identifiers, foreign keys, check constraints, and indexes on case_id, evidence_id, status, created_at, and rule_id as appropriate.
- Keep original extracted values immutable. Store reviewer corrections separately or as versioned correction fields; never overwrite provenance without an audit record.

- Use decimal/numeric types for quantities and money rather than binary floating point. Store currency, unit, rounding policy, and conversion source explicitly.
- Record model name/version, prompt version, schema version, OCR processor/version, rule version, and report template version for reproducibility.
- Apply tenant scoping to all reads and writes. Authorization should be enforced in the service layer and, if practical, reinforced with PostgreSQL row-level security.
- Use append-only review/audit events. Restrict database roles so application runtime cannot casually alter or delete audit history.

6. API surface (illustrative)

Method	Endpoint	Purpose
POST	/v1/cases	Create a case after role and tenant checks.
GET	/v1/cases/{case_id}	Return case metadata and authorized summary.
POST	/v1/cases/{case_id}/uploads	Create an upload session and return a short-lived signed URL plus evidence ID.
POST	/v1/evidence/{evidence_id}/complete	Verify uploaded object and enqueue processing.
GET	/v1/cases/{case_id}/status	Return per-file and case-level processing status.
GET	/v1/cases/{case_id}/findings	Return findings, cited claims, grounding state, and review state.
GET	/v1/evidence/{evidence_id}/pages/{page}	Return authorized page text/anchor metadata or a short-lived view URL.
POST	/v1/findings/{finding_id}/review-actions	Record confirm/correct/dismiss/note action with validation and audit event.
POST	/v1/cases/{case_id}/reports	Queue report generation; return job/report status.
GET	/v1/reports/{report_id}/download	Authorize access and issue a short-lived signed download URL.

The endpoint list is illustrative, not a finalized OpenAPI contract. Use versioned routes, consistent error envelopes, pagination, request IDs, and idempotency keys for operations that may be retried.

7. AI, OCR, grounding, and reconciliation design

7.1 Separate extraction from inference

The model should identify candidate fields and quote supporting text; it should not calculate the reconciliation result. Use a strict response schema with explicit null/unknown states. A representative claim includes field_name, raw_value, normalized_value, source_quote, page_hint, and extraction_status. Application validation must reject malformed values and unsupported types.

7.2 Grounding is more than quote matching

- Normalize only safe differences such as whitespace and line-wrap artifacts; preserve the original source text and the normalization procedure.
- Match within the claimed page first. If multiple matches exist, store ambiguity and require review rather than arbitrarily selecting a location.
- A quote match proves that the quoted string appears in extracted text; it does not prove that the model interpreted the quote correctly or that the document itself is authentic.

- For tables, preserve row/column context and neighboring labels. A number such as “80” is not meaningful without its field label, unit, date, and document context.
- Use bounding boxes for source highlighting, but retain text-span and block IDs because OCR geometry and PDF viewer coordinate systems may differ.

7.3 Deterministic rule engine

Represent each reconciliation as a versioned rule with declared inputs, preconditions, unit semantics, calculation, and output. Example: ordered_quantity = 100 and delivered_quantity = 80 yields shortfall = max(0, 100 − 80) = 20 units. The rule engine should reject incompatible units or missing inputs and emit a data-quality issue rather than silently coercing them.

A “promise with no follow-up delivery note” should be described as “no matching delivery evidence found in the documents reviewed,” not as proof that delivery never occurred. Findings must distinguish absence of evidence from evidence of absence.

7.4 Prompt injection and model boundaries

- Treat every uploaded document and OCR output as untrusted data, never as instructions.
- Do not attach privileged tools to extraction or explanation model calls. Keep tools, secrets, and authorization decisions outside model context.
- Constrain outputs with schema validation, maximum lengths, allowlisted fields, enum checks, and safe parsing; never execute model-generated code or SQL.
- Send only the minimum relevant text and redact unnecessary personal data when feasible. Establish retention rules for prompts and model traces.
- Use a separate explanation call only when it adds value; otherwise render a deterministic template to reduce cost and variation.

8. Security, privacy, and governance

Control area	Implementation guidance
Identity	Verify Firebase ID tokens using firebase-admin; validate issuer, audience, expiry, and revocation policy. App Check is defense in depth, not authorization.
Authorization	Perform case-level and tenant-level permission checks on every API route and object access. Avoid relying on unguessable IDs as access control.
Network and ingress	Restrict Cloud Run ingress where feasible; require authenticated service-to-service calls for worker endpoints. Use TLS in transit and private connectivity to Cloud SQL where supported by the deployment design.
Storage	Private buckets, uniform bucket-level access, public access prevention, least-privilege service accounts, lifecycle policies, and short signed-URL expiry. Avoid logging signed URLs.
Secrets	Store database credentials and any signing configuration in Secret Manager. Prefer workload identity/service accounts over long-lived service-account keys.
Database	Use separate database roles for migrations and runtime, least privilege, encrypted backups, tested restoration, connection pooling, and controlled schema migrations.
Audit	Record review changes, access-sensitive actions, report creation/download events where required, and administrative changes. Define retention and tamper-resistance requirements.
Data residency	Singapore is a proposed deployment region, not a

	guarantee that every dependent service, log, backup, model endpoint, or support path remains there. Confirm product-specific regional processing and contractual requirements.
Compliance	Perform a data classification and privacy impact assessment. Define lawful purpose, retention/deletion, access requests, breach handling, and any sector-specific obligations before using real case documents.

9. Reliability, scaling, and operations

- Use Pub/Sub retry policies and a dead-letter topic. Make the worker idempotent using a stable job ID and unique constraints so duplicate delivery does not duplicate claims or findings.
- Persist explicit statuses such as uploaded, queued, extracting, grounding, reconciling, needs_review, completed, failed_retryable, and failed_terminal.
- Use bounded retries with exponential backoff for transient service failures. Do not retry invalid files, unsupported formats, or schema failures indefinitely.
- Set Cloud Run request/instance concurrency based on memory-heavy OCR/PDF workloads; benchmark rather than assuming the default is appropriate.
- Use Cloud SQL connection pooling and size the connection budget against Cloud Run maximum instances. Consider the Cloud SQL connector and private IP configuration appropriate to the environment.
- Instrument structured logs with case_id, evidence_id, job_id, trace/correlation ID, model version, and rule version; exclude raw sensitive document text and secrets.
- Create dashboards and alerts for queue age, processing latency, worker failure rate, OCR/model errors, database connections, storage errors, and cost anomalies.
- Use min instances = 0 as a cost-saving starting point for a hackathon, accepting cold starts. Do not use it if latency commitments require warm instances.
- Back up Cloud SQL, define recovery point/recovery time objectives, and test restore procedures rather than relying only on backup configuration.

10. Build, release, and environments

Use separate development, staging, and production projects or equivalent isolation boundaries. For a hackathon, at minimum separate configuration and service accounts by environment, and avoid placing production-like sensitive documents in development.

- Cloud Build trigger runs linting, unit tests, type/schema tests, dependency checks, and container build.
- Push immutable version-tagged images to Artifact Registry; deploy the API and worker as separate Cloud Run services.
- Apply database migrations as a controlled release step using a migration identity, not automatically from every application instance.
- Store environment-specific configuration in Secret Manager and runtime configuration variables; never commit credentials or private keys.
- Use staged rollout or a rollback strategy for Cloud Run revisions. Record the source commit and image digest for each deployment.
- Flutter web can be hosted on Firebase Hosting; configure appropriate cache headers, security headers, and API origin settings. Mobile release signing remains a separate platform process.

11. Testing and acceptance criteria

Test area	Acceptance criterion
Core arithmetic	Given ordered=100 and delivered=80 in the same unit, the rule outputs shortfall=20; no LLM arithmetic is used.
Grounding	A claim with an exact unique quote maps to a source page/span; absent or ambiguous quotes are flagged for review.
Authorization	A user who is not a member of a case cannot read its metadata, evidence, findings, report, or signed URL.
Upload integrity	Oversized, mismatched, unsupported, or incomplete uploads are rejected or quarantined; the server validates object metadata.
Idempotency	Redelivery of the same Pub/Sub message does not create duplicate findings or audit actions.
Review audit	Every correction/confirmation/dismissal records actor, timestamp, previous/new values, reason where required, and target IDs.
Report traceability	Export includes finding IDs, cited evidence references, review status, generation time, and an explanation that findings require human interpretation.
Failure handling	OCR/model timeouts and transient database failures are retried within bounds and eventually become visible failed states.
Security	Signed URLs expire, buckets are not public, secrets are not in source/logs, and logs do not contain raw sensitive document content by default.
Demo vertical slice	Create case → upload five synthetic documents → extract evidence → flag 20-unit gap → open source highlight → review finding → export PDF.

12. Implementation roadmap

Phase 1 — vertical slice

Flutter sign-in and case creation; FastAPI auth/authorization; private bucket and signed uploads; synthetic sample documents; PostgreSQL schema; one deterministic shortfall rule; findings UI; simple PDF export.

Phase 2 — robust ingestion

Pub/Sub worker, idempotency, Document AI OCR, digital PDF/DOCX extraction, normalized page/block model, processing states, retry and dead-letter handling.

Phase 3 — evidence-grounded AI

Gemini schema-constrained extraction, quote matching, ambiguity handling, field normalization, model/prompt versioning, and rule-driven explanations.

Phase 4 — review and audit

PDF overlays, reviewer corrections and notes, immutable audit events, permissions tests, report history, export security.

Phase 5 — production hardening

Cloud Monitoring dashboards, alerts, cost budgets, backup/restore drills, load tests, threat modeling, retention/deletion, deployment rollback.

Optional — read-only case assistant

ADK-based Q&A over narrow read-only API tools. Return citations to finding/claim IDs and source pages; prohibit model-controlled writes and consequential decisions.

13. Cost and deployment considerations

The principal variable costs are Document AI page processing, Vertex AI model tokens, Cloud Run CPU/memory and request time, Cloud SQL baseline capacity, Cloud Storage, Pub/Sub, logging, and network egress. Actual cost depends on page volume, model choice, document size, region, retention, and concurrency; do not publish a cost estimate until these assumptions are measured.

- Use budget alerts and per-service labels from the first deployment; budgets alert but do not necessarily stop spending.
- Set Cloud Run min instances to zero for low-volume demonstrations and limit maximum instances to protect downstream services and cost.
- Choose the smallest Cloud SQL tier that meets memory, storage, connection, and performance needs; test connection pooling before increasing instances.
- Cache or reuse OCR results when documents are unchanged; avoid repeated model calls during review.
- Keep prompt context narrow, cap page/text length, and use deterministic templates when an LLM explanation is unnecessary.
- Use synthetic documents for demos and tests. Redact or avoid personal data unless there is a documented need and approved handling process.

14. Risks and mitigations

Risk	Mitigation
OCR errors / poor scans	Show OCR uncertainty, preserve source image, allow correction, test representative scans, and route low-quality pages to manual review.
False confidence from quote matching	Explain that grounding verifies text presence, not semantic correctness or document authenticity; retain label/context and require human review.
Prompt injection in evidence	Treat OCR as data, not instructions; no privileged tools in extraction calls; schema validation and strict allowlists.
Duplicate or lost jobs	Transactional outbox, idempotency keys, retry policy, dead-letter topic, and periodic reconciliation of queued DB jobs against Pub/Sub/worker state.
Region/service mismatch	Verify Document AI processor availability, Vertex AI model availability, Cloud Run/SQL capabilities, and data-processing terms before finalizing region.
Costs exceed hackathon budget	Budget alerts, quotas where available, max instances, small test batches, token/page limits, and explicit cleanup of test resources.
Unauthorized access via signed URLs	Short expiry, object-specific URLs, authorization before issuance, no URL logging, and no public bucket access.
Audit trail can be altered	Separate runtime and migration roles, append-only event model, restricted admin access, and retention/export of required audit records.

15. Recommended technology stack

Layer	Recommended tools
Client	Flutter / Dart; Flutter web; a PDF viewer that supports page

	navigation and overlay rendering; Firebase Hosting for web.
API	Python 3.x; FastAPI; Pydantic; firebase-admin; SQLAlchemy 2.x; Alembic; pytest; Ruff; mypy or pyright as appropriate.
Persistence	Cloud SQL for PostgreSQL; Cloud Storage; Pub/Sub; optionally Memorystore only if a measured use case emerges.
Document processing	Native PDF text extraction library; DOCX parser; Document AI OCR for scans/images; canonical page/block representation.
AI	Vertex AI Gemini for extraction and constrained explanations; JSON schema validation; prompt/schema versioning; optional Google ADK only for read-only Q&A.
Infrastructure	Cloud Run API and worker; Cloud Build; Artifact Registry; Secret Manager; IAM; Cloud Logging; Cloud Monitoring; Cloud Audit Logs.
Reports	HTML templates rendered to PDF using WeasyPrint in a worker image, after validating system package dependencies and font availability.
Quality	Unit and integration tests; synthetic golden-document corpus; regression checks for extracted fields, grounding, rule outputs, authorization, and report content.

16. Key design decisions and open questions

- Confirm data residency and contractual requirements for the documents and users before choosing Singapore as the production region.
- Confirm exact Gemini model IDs and Document AI processor availability in asia-southeast1 at implementation time; use configuration, not hard-coded model names.
- Define supported file types, file size/page limits, malware scanning or quarantine needs, and document retention/deletion rules.
- Define tenant model, reviewer roles, who can export, whether a second reviewer is required for high-impact findings, and whether dismissed findings remain visible in reports.
- Decide whether PDF rendering and heavy OCR share one worker image or use separate Cloud Run worker services based on memory, latency, and dependency isolation.
- Define how confidence is calibrated and displayed. Do not present model-provided confidence as a statistically calibrated probability unless validated.
- Determine whether the competition requires an explicit agentic component; if so, implement read-only case Q&A as an optional bounded feature, not as a dependency of the core workflow.

17. Conclusion

This architecture provides a practical path from a hackathon demonstration to a controlled production service. The critical engineering choice is the separation of evidence extraction, grounding, deterministic reconciliation, and human decision-making. GCP managed services reduce infrastructure burden, while the asynchronous worker design isolates slow document processing from user-facing requests. ProofLoop should be judged not by whether its AI sounds confident, but by whether each finding is reproducible, source-linked, permission-protected, reviewable, and auditable.

Appendix A — Suggested status model

Case: draft → processing → review_required → reviewed → report_ready (with failed or archived states as needed). Evidence: uploaded → queued → extracting → grounding → reconciling → completed, needs_review, failed_retryable, or failed_terminal. Finding review: open → confirmed, corrected, or dismissed. Preserve transitions and actors in the audit trail.

Appendix B — Implementation caveats

This is a reference design, not a claim that every service feature, model, processor, or regional configuration has been live-verified. Before deployment, verify current GCP product documentation, service quotas, regional availability, model lifecycle, pricing, and contractual data-processing terms. Pin dependency versions, run security review, and validate the design with representative synthetic documents before handling real data.